Universidade de Coimbra
Faculdade de Ciências e Tecnologia
Departamento de Engenharia Informática

Sistemas de Gestão de Dados

Projeto 1 - Benchmark

SSB

Docente

Pedro Nuno San-Bento Furtado

Equipa

Carlos João Lopes Mendes, nº2017257272

Filipe Miguel Fonseca dos Santos, nº2017271196

Índice

Autoavaliação do projeto 3
Introdução 4
Load de Dados 4
Search Without Keys 5
Key Creation Time 7
Search With Keys 9
Query Execution Plans 10
Conclusão 13
Dificuldades Encontradas 14
Anexo 14

Autoavaliação do projeto

a. Lists of what the group succeeded to do and what is missing

-O grupo conseguiu atingir os objetivos básicos que eram pedidos no enunciado, não havendo em princípio nenhum aspeto que esteja em falta.

b. self-evaluation of the group (0-100%)

-90%

c. List of what each student contributed (no repetitions)

-Para o nosso projeto, decidimos realizar todo o trabalho em conjunto, através de videochamadas nas quais um elemento partilhou o ecrã e o outro auxiliou a pensar na solução e a encontrar erros. Temos os tempos de chamada registados e podemos associá-los ao que fizemos em cada uma delas, mencionando quem nessa chamada partilhou o ecrã e quem ficou a auxiliar.

Chamada	Tempo	Tema
1	35min e 8s	Discussão sobre dúvidas referentes ao projeto.
2	2h e 20min	Compilação dos dados recolhidos e criação de gráficos
3	55min e 41s	Formatação dos dados para apresentação e início do relatório
4	1h e 3min	Continuação do relatório
5	42min e 26s	Continuação do relatório
6	1h e 30min	Continuação do relatório
7	1h e 1min	Conclusão do relatório
8	40min	Correção do relatório com base nas sugestões do professor

d. self-evaluation of each student in the group (0-100%)

Carlos Mendes: 90%
Filipe Santos: 90%

e. hours of effort by each student separately

Neste caso, como em vez de dividirmos tarefas de forma autónoma, fizemos tudo em conjunto, as horas de esforço são as mesmas.

Horas de esforço = ~8 horas e 46 minutos (Não inclui tempo usado a correr queries)

Introdução

Neste projeto procedeu-se à análise de performance ao carregar e realizar queries sobre um Star Schema Benchmark (SSB) numa base de dados postgreSQL.

Neste relatório pretendemos mostrar os dados recolhidos para esta análise, bem como as conclusões que tirámos através destes.

Devido a otimizações do próprio sistema de base de dados (ex: cache), não nos foi possível fazer uma análise completa, pois a execução das diferentes queries foram afetadas por dados já existentes em cache. Num cenário ideal, deveríamos ter corrido cada query diversas vezes com o sistema a cold, mas uma vez que colocar o sistema a cold implica reiniciar o sistema (computador), optámos por omitir este passo. No entanto para estudar a diferença de performance entre o sistema a cold e a hot, testámos a query (Q1.1) tanto a hot como a cold. Para todas as outras queries, optámos por corrê-las uma única vez em estado hot.

Durante os testes, apercebemo-nos que devido a especificações das máquinas (disco, processador, …) em que corríamos cada query (computadores dos elementos do grupo) obtivemos tempos de processamento consideravelmente diferentes. Devido a esta diferença, cada membro do grupo correu as quereis de forma a podermos estudar o impacto que as especificações das máquinas podem ter.

Máquina	Sistema Operativo	Disco	Memória	Processador
Carlos	Windows 10	SSD	16 GB	Intel Core i7
Filipe	Windows 10	HDD	8 GB	Intel Core i3

Tabela 1 – Especificações das máquinas dos elementos do grupo.

Load de Dados

Na tabela 2 podemos ver os dados recolhidos, sendo observável que, como esperado, o tempo necessário de execução aumenta conforme o tamanho dos ficheiros de origem das tabelas, tanto em relação ao número de rows como à quantidade de MegaBytes (MB) ocupados.

Pelos dados obtidos, podemos observar que a velocidade de criação das tabelas foi semelhante entre as tabelas (valores próximos da média). Nota-se, no entanto, uma velocidade significativamente menor na criação da tabela “date”, quer no número de linhas produzidas, quer no número de MB lidos. Isto pode dever-se a operações efetuadas antes de começar a ler o ficheiro de origem em si (overhead), que, devido ao tamanho pequeno da tabela, tiveram um impacto maior no tempo de processamento do que nas outras tabelas.

Outra observação que podemos fazer em relação aos dados obtidos é o impacto que as especificações da máquina em que as queries são executadas têm no tempo de processamento. Podemos ver que a máquina com o disco SSD teve um desempenho superior em relação à máquina com o disco HDD, tendo obtido tempos de processamento inferiores. Tendo em conta que as operações de load resumem-se sobretudo a fazer “fetch” de dados em

memória, consideramos que o disco da máquina tem um peso maior nos resultados obtidos, em relação a outros fatores como o processador.

	Tabelas	customer	date	lineorder	part	supplier	Mean
	Número de Rows	450000	2557	89987410	800000	30000	-
	Tamanho (MB)	42,471	0,227	9189,684	68,056	2,505	-
Filipe (HDD, i3)	Tempos de Load (s)	8,57	**0,236**	1457	9,969	0,453	-
	Tempo de Load	8 s 570 ms	236 ms	24 min 17 seg	9 secs 969 ms	453 ms	-
	Rows/s	52508,75146	**10834,74576**	61762,1208	80248,77119	66225,16556	**54315,91095**
	MB/s	4,955775963	**0,961864407**	6,307264242	6,826762965	5,529801325	**4,91629378**
Carlos (SDD, i7)	Tempos de Load (s)	1,857	**0,072**	403	3,077	0,178	-
	Tempo de Load	1 s 857 ms	72 ms	6 min 43 s	3 s 77 ms	178 ms	-
	Rows/s	242326,3328	**35513,88889**	223293,8213	259993,5002	168539,3258	**185933,3738**
	MB/s	22,87075929	**3,152777778**	22,8031861	22,11764706	14,07303371	**17,00348079**

Tabela 2 – Dados de loading das diferentes tabelas.

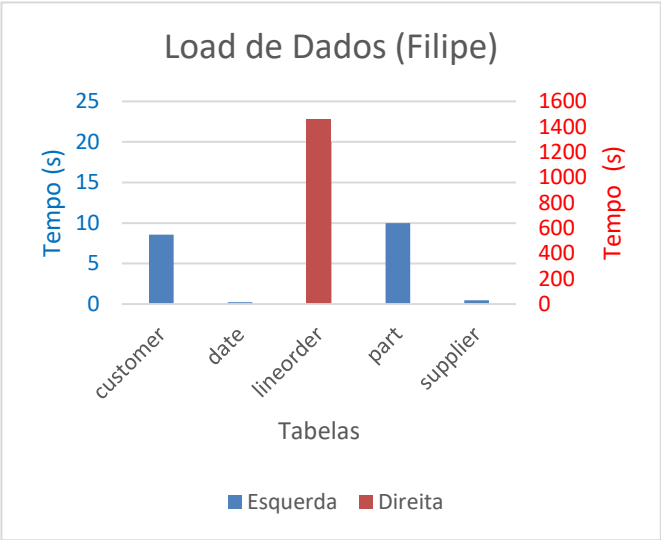

Figura 2 – Gráfico dos tempos de load das tabelas.

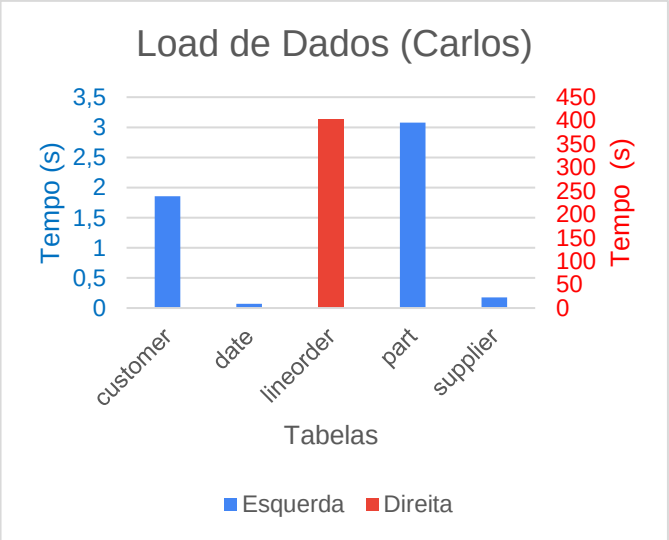

Figura 1 - Gráfico dos tempos de load das tabelas.

Search Without Keys

A partir dos dados recolhidos podemos observar que, embora todas as queries envolvam acessos às várias tabelas criadas, em particular à lineorder, os tempos obtidos foram consideravelmente inferiores ao tempo de criação da tabela lineorder. Isto leva-nos a concluir que as operações de criação de tabelas e seu preenchimento são muito mais pesadas do que as operações de pesquisa dos dados.

Tal como ocorreu anteriormente, as especificações das máquinas em que corremos as queries tiveram um forte impacto nas pesquisas efetuadas. Neste cenário, como as pesquisas já envolveram uma grande quantidade de operações lógicas, podemos dizer que fatores como o processador levaram a uma maior diferença entre a performance das máquinas testadas do que aquela observada durante a criação dos dados. No entanto, apesar das diferenças entre

especificações das máquinas, podemos observar que cada query teve um tempo de execução relativamente às outras queries que foi semelhante entre máquinas. Isto pode ser observado através da linha vermelha de cada gráfico.

Podemos observar através dos gráficos que, em geral, as queries Q2.1, Q3.1 e Q4.1 foram as que demoraram mais tempo. Uma vez que as queries de cada grupo apenas apresentam ligeiras diferenças entre si, considerámos a hipótese de que otimizações da base de dados como a cache levaram a que queries do mesmo grupo tivessem um tempo de processamento inferior à primeira de cada grupo. No entanto, ao correr as queries Q4.3, Q4.2 e Q4.1 (por esta ordem), observámos que esta hipótese não era válida, já que obtivemos tempos de processamento com diferenças semelhantes às observáveis na tabela. (~14s, ~20s, ~22s, respetivamente). Sendo assim, consideramos mais provável que as queries referidas inicialmente sejam simplesmente as mais pesadas.

É de notar que, em ambas as máquinas, as queries foram processadas pela ordem apresentada no gráfico.

Para testar as diferenças entre correr queries com o sistema a cold e a hot, decidimos correr a query Q1.1 em ambas as condições. Como podemos ver pelos valores obtidos, ter o sistema a cold levou a que as queries demorassem muito mais tempo a correr (2-4x mais).

Filipe (HDD, i3)				
Query	Tempo (s) - Hot	Tempo - Hot	Tempo (s) - Cold	Tempo - Cold
1_1	262	4 min 22s	506	8 min 26 s
1_2	212	3 min 32 s		
1_3	229	3 min 49 s		
2_1	245	4 min 5 s		
2_2	220	3 min 40 s		
2_3	274	4 min 34 s		
3_1	311	5 min 11 s		
3_2	222	3 min 42 s		
3_3	233	3 min 53 s		
3_4	228	3 min 48 s		
4_1	348	5 min 48 s		
4_2	291	4 min 51 s		
4_3	238	3 min 58 s		

Tabela 3 – Tempo de execução das diferentes queries. (ver anexo para lista de queries)

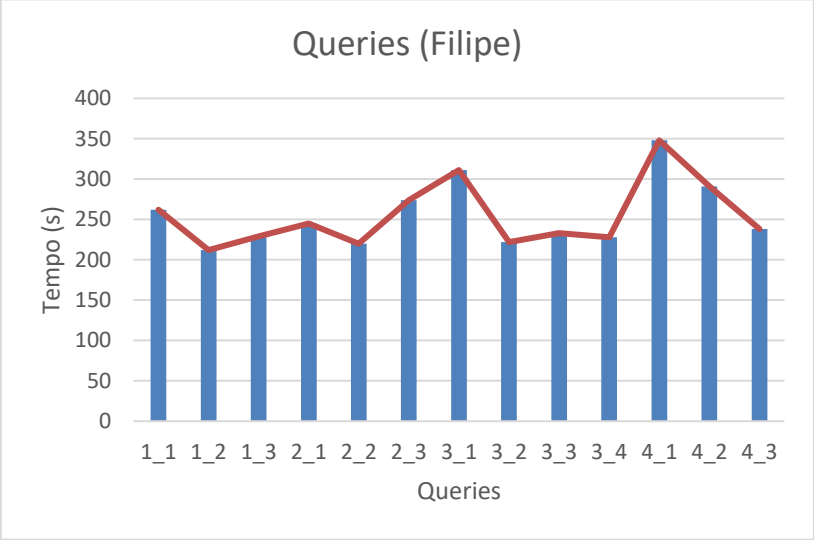

Figura 3 – Gráfico dos tempos de execução das diferentes queries.

Carlos (SSD, i7)				
Query	Tempo (s) - Hot	Tempo - Hot	Tempo (s) - Cold	Tempo - Cold
1_1	11,528	11s 528ms	43,633	43s 633ms
1_2	12,081	12s 81ms		
1_3	11,306	11s 306ms		
2_1	14,283	14s 283ms		
2_2	11,472	11s 472ms		
2_3	10,602	10s 602ms		
3_1	16,582	16s 582ms		
3_2	11,482	11s 482ms		
3_3	9,868	9s 868ms		
3_4	10,716	10s 716ms		
4_1	17,25	17s 250ms		
4_2	15,921	15s 921ms		
4_3	13,502	13s 502ms		

Tabela 4 - Tempo de execução das diferentes queries. (ver anexo para lista de queries)

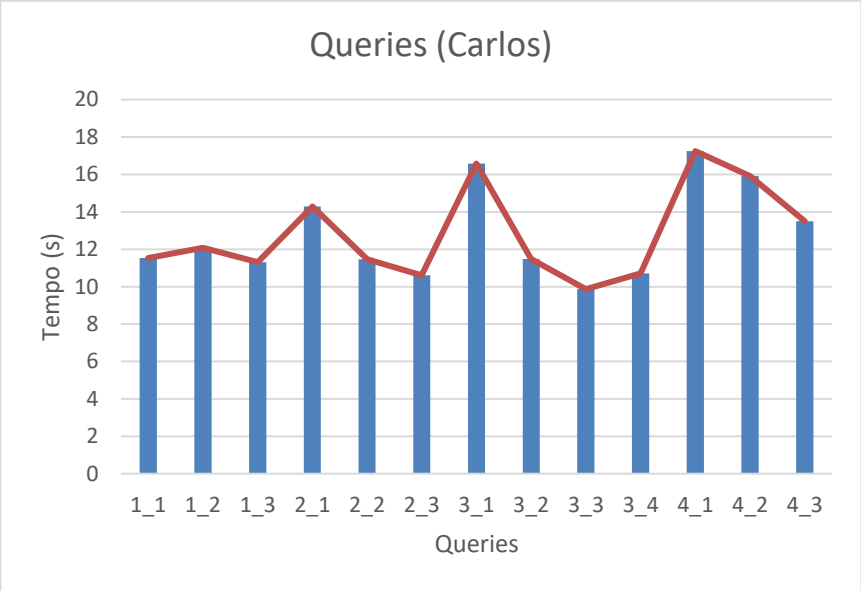

Figura 4 - Gráfico dos tempos de execução das diferentes queries.

Key Creation Time

Durante a criação de chaves nas diferentes tabelas podemos observar novamente a diferença de tempo provocada pelas especificações das máquinas, por razões semelhantes às mencionadas no cenário de load dos dados.

Pelos dados obtidos, podemos observar que as operações sobre a tabela lineorder (keys 5-10) demoraram consideravelmente mais do que as restantes. Podemos observar ainda que a operação 5, que envolve a criação de uma primary key nesta tabela demorou significativamente mais tempo do que as queries 6-10, que envolveram a criação de foreign keys. A diferença entre as os grupos 1-4 e 5-10 era espectável devido ao grande tamanho da tabela lineorder relativamente às restantes. A diferença entre o tempo de criação da primary key e das foreign keys pode ser explicada devido ao facto de que, ao adicionar a primary key, a base de dados ainda tem de verificar que não existem duas keys iguais, enquanto que as foreign keys servem de ligação entre linhas de diferentes tabelas, não havendo esta verificação.

Filipe (HDD, i3)		
Keys	Tempos de Criação (s)	Tempos de Criação
1	0,683	683 ms
2	0,297	297 ms
3	4,023	4 s 23 ms
4	4,204	4 s 204 ms
5	379	6 min 19 s
6	132	2 min 12 s
7	139	2 min 19 s
8	142	2 min 22 s
9	142	2 min 22 s
10	147	2 min 27s

Tabela 6 – Tempo de criação das keys (ver anexo para lista de keys)

Carlos (SSD, i7)		
Keys	Tempos de Criação (s)	Tempos de Criação
1	0,076	76ms
2	0,076	76ms
3	0,378	378ms
4	0,592	592ms
5	81	1min 21s
6	26,827	26s 827ms
7	24,7	24s 700ms
8	23,86	23s 860ms
9	24,852	24s 852ms
10	39,49	39s 49ms

Tabela 5 - Tempo de criação das keys (ver anexo para lista de keys)

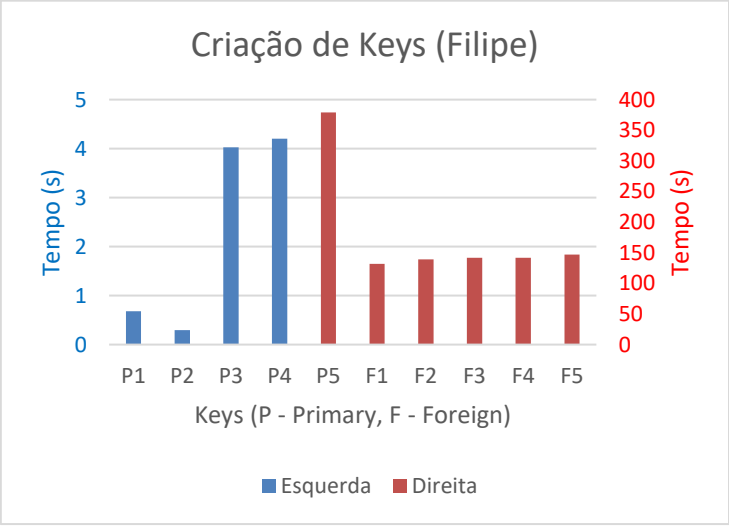

Figura 6 - Gráfico dos tempos de criação das keys

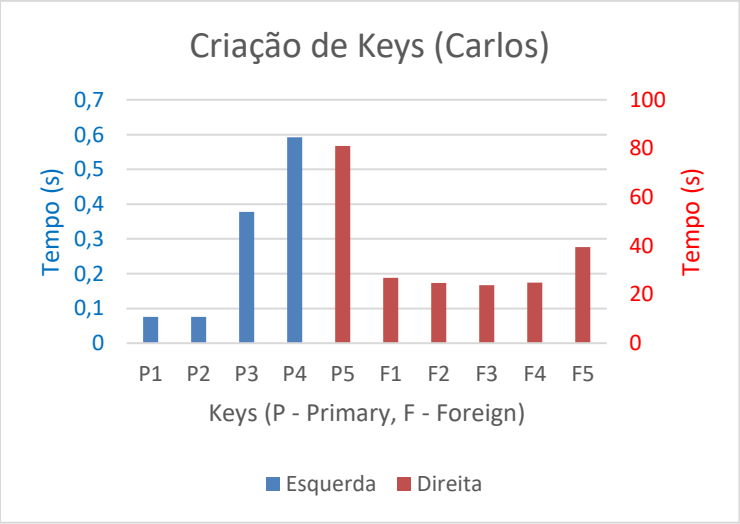

Figura 5 - Gráfico dos tempos de criação das keys

Search With Keys

Pelos dados obtidos podemos observar que a adição de keys às tabelas, de um modo geral, melhorou ligeiramente o desempenho das pesquisas, já que os tempos obtidos foram, em geral, menores do que aquilo que foram com as pesquisas sem o uso de keys. Porém, as 4 Queries que analisámos com o plano de execução não utilizam as keys, mas sim sequential scan. Isto pode significar que, na verdade, as diferenças observadas nas tabelas 7 e 8 se devem a outros aspetos da base de dados (ex: otimizações da cache) do que ao uso de keys.

Nos dados apresentados, temos um valor claramente fora da gama dos restantes (Q2.1 - Filipe), sendo que consideramos esse valor como um outlier. Ao voltar a correr esta query em particular obteve-se o tempo de execução de 3min e 38s. Isto mostra que foi realmente um outlier e não algo que se deva à query ou ao sistema usado para a correr.

Filipe (HDD, i3)			
Query	Tempo (s)	Tempo	Diferença sem Keys
1_1	193	3 min 13 s	-69
1_2	213	3 min 33s	1
1_3	248	4 min 8 s	19
2_1	436	**7 min 16 s**	191
2_2	229	3 min 49 s	9
2_3	212	3 min 32 s	-62
3_1	232	4 min 52 s	-79
3_2	227	3 min 47 s	5
3_3	227	3 min 47 s	-6
3_4	233	3 min 53 s	5
4_1	306	5 min 6 s	-42
4_2	271	4 min 31 s	-20
4_3	223	3 min 43 s	-15

Tabela 7 - Tempo de execução das diferentes queries. (Diferença = TempoComKeys - TempoSemKeys)

Carlos (SSD, i7)			
Query	Tempo (s)	Tempo	Diferença sem Keys
1_1	11,386	11s 386ms	-0,142
1_2	11,337	11s 337ms	-0,744
1_3	10,608	10s 608ms	-0,698
2_1	13,851	13s 851ms	-0,432
2_2	11,464	11s 464ms	-0,008
2_3	10,027	10s 27ms	-0,575
3_1	16,704	16s 704ms	0,122
3_2	11,444	11s 444ms	-0,038
3_3	10,004	10s 4ms	0,136
3_4	9,868	9s 868ms	-0,848
4_1	16,601	16s 601ms	-0,649
4_2	15,884	15s 884ms	-0,037
4_3	11,401	11s 401ms	-2,101

Tabela 8 - Tempo de execução das diferentes queries. (Diferença = TempoComKeys - TempoSemKeys)

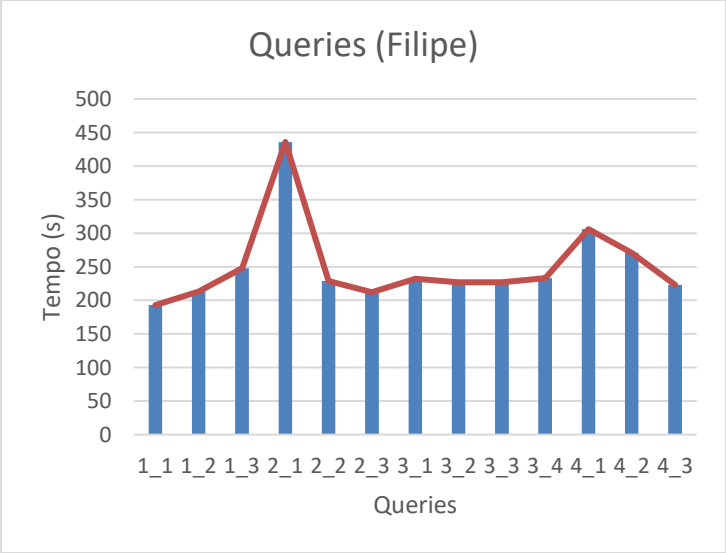

Figura 7 - Gráfico dos tempos de execução das diferentes queries.

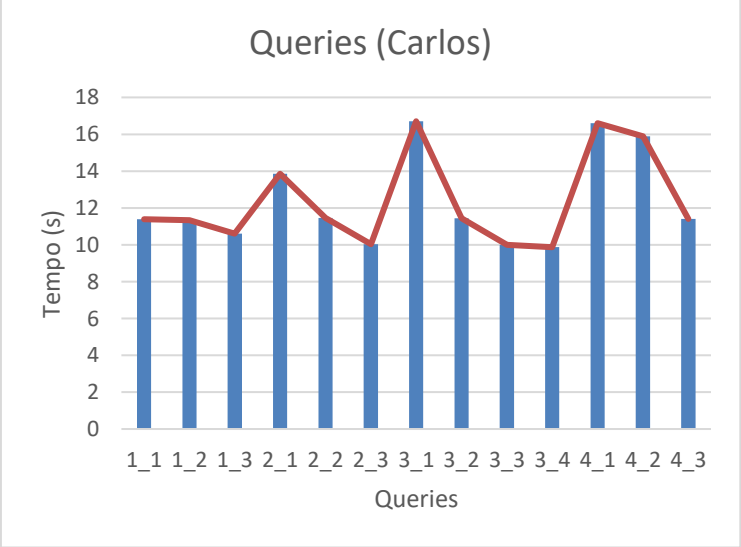

Figura 8 - Gráfico dos tempos de execução das diferentes queries.

Query Execution Plans

Após a análise dos planos de execução das diferentes queries, podemos observar que as decisões que a base de dados podia tomar relativamente ao processamento das tabelas foram semelhantes entre elas. De notar, em especial, que a leitura das tabelas foi feita através de sequential scans (leitura completa da tabela) e os joins foram feitos através de Hash Join (o que é a escolha normalmente usada para tabelas grandes como o lineorder). Sendo que os sequential scans foram as escolhas tomadas, podemos concluir que as keys não foram utilizadas, pelo que o custo calculado pelo motor de base de dados para a utilização de keys deve ser superior ao do sequential scan.

Optámos também por comparar os planos de execução devolvidos em cada máquina. No entanto, os planos eram iguais, com exceção do cálculo do custo, que apresentou resultados praticamente idênticos. Sendo assim, em baixo apresentamos apenas os planos de execução de uma das máquinas.

Data Output | Explain | Messages | Notifications

	QUERY PLAN text
1	Finalize Aggregate (cost=1873243.13..1873243.14 rows=1 width=8)
2	-> Gather (cost=1873242.91..1873243.12 rows=2 width=8)
3	Workers Planned: 2
4	-> Partial Aggregate (cost=1872242.91..1872242.92 rows=1 width=8)
5	-> Hash Join (cost=75.53..1870503.04 rows=695951 width=4)
6	Hash Cond: (lineorder.lo_orderdate = date.d_datekey)
7	-> Parallel Seq Scan on lineorder (cost=0.00..1857608.18 rows=4875471 width=8)
8	Filter: ((lo_discount >= 1) AND (lo_discount <= 3) AND (lo_quantity < 25))
9	-> Hash (cost=70.96..70.96 rows=365 width=4)
10	-> Seq Scan on date (cost=0.00..70.96 rows=365 width=4)
11	Filter: (d_year = 1993)

Figura 9 – Plano de execução da query Q1.1

	QUERY PLAN text
1	Finalize GroupAggregate (cost=1708584.56..1708794.56 rows=7000 width=21)
2	Group Key: date.d_year, part.p_brand1
3	-> Sort (cost=1708584.56..1708619.56 rows=14000 width=21)
4	Sort Key: date.d_year, part.p_brand1
5	-> Gather (cost=1706150.44..1707620.44 rows=14000 width=21)
6	Workers Planned: 2
7	-> Partial HashAggregate (cost=1705150.44..1705220.44 rows=7000 width=21)
8	Group Key: date.d_year, part.p_brand1
9	-> Hash Left Join (cost=23165.27..1702827.52 rows=309723 width=17)
10	Hash Cond: (lineorder.lo_orderdate = date.d_datekey)
11	-> Hash Join (cost=23068.74..1701916.62 rows=309723 width=17)
12	Hash Cond: (lineorder.lo_suppkey = supplier.s_suppkey)
13	-> Hash Join (cost=22187.00..1697008.78 rows=1533535 width=21)
14	Hash Cond: (lineorder.lo_partkey = part.p_partkey)
15	-> Parallel Seq Scan on lineorder (cost=0.00..1576397.53 rows=37494753 width=16)
16	-> Hash (cost=21778.00..21778.00 rows=32720 width=13)
17	-> Seq Scan on part (cost=0.00..21778.00 rows=32720 width=13)
18	Filter: ((p_category)::text = 'MFGR#12'::text)
19	-> Hash (cost=806.00..806.00 rows=6059 width=4)
20	-> Seq Scan on supplier (cost=0.00..806.00 rows=6059 width=4)
21	Filter: ((s_region)::text = 'AMERICA'::text)
22	-> Hash (cost=64.57..64.57 rows=2557 width=8)
23	-> Seq Scan on date (cost=0.00..64.57 rows=2557 width=8)

Figura 10 - Plano de execução da query Q2.1

Data Output | Explain | Messages | Notifications

	QUERY PLAN text
1	Sort (cost=1818393.99..1818404.92 rows=4375 width=28)
2	Sort Key: date.d_year, (sum(lineorder.lo_revenue)) DESC
3	-> Finalize GroupAggregate (cost=1817976.28..1818129.41 rows=4375 width=28)
4	Group Key: customer.c_nation, supplier.s_nation, date.d_year
5	-> Sort (cost=1817976.28..1817998.16 rows=8750 width=28)
6	Sort Key: customer.c_nation, supplier.s_nation, date.d_year
7	-> Gather (cost=1816484.62..1817403.37 rows=8750 width=28)
8	Workers Planned: 2
9	-> Partial HashAggregate (cost=1815484.62..1815528.37 rows=4375 width=28)
10	Group Key: customer.c_nation, supplier.s_nation, date.d_year
11	-> Hash Join (cost=15176.75..1802662.69 rows=1282193 width=24)
12	Hash Cond: (lineorder.lo_orderdate = date.d_datekey)
13	-> Hash Join (cost=15072.00..1798625.22 rows=1495698 width=24)
14	Hash Cond: (lineorder.lo_custkey = customer.c_custkey)
15	-> Hash Join (cost=881.50..1675716.55 rows=7548944 width=20)
16	Hash Cond: (lineorder.lo_suppkey = supplier.s_suppkey)
17	-> Parallel Seq Scan on lineorder (cost=0.00..1576397.53 rows=37494753 width=16)
18	-> Hash (cost=806.00..806.00 rows=6040 width=12)
19	-> Seq Scan on supplier (cost=0.00..806.00 rows=6040 width=12)
20	Filter: ((s_region)::text = 'ASIA'::text)
21	-> Hash (cost=12640.00..12640.00 rows=89160 width=12)
22	-> Seq Scan on customer (cost=0.00..12640.00 rows=89160 width=12)
23	Filter: ((c_region)::text = 'ASIA'::text)
24	-> Hash (cost=77.35..77.35 rows=2192 width=8)
25	-> Seq Scan on date (cost=0.00..77.35 rows=2192 width=8)
26	Filter: ((d_year >= 1992) AND (d_year <= 1997))

Figura 11 - Plano de execução da query Q3.1

Data Output | Explain | Messages | Notifications

	QUERY PLAN text
1	Finalize GroupAggregate (cost=1858987.04..1858993.61 rows=175 width=20)
2	Group Key: date.d_year, customer.c_nation
3	-> Sort (cost=1858987.04..1858987.92 rows=350 width=28)
4	Sort Key: date.d_year, customer.c_nation
5	-> Gather (cost=1858935.50..1858972.25 rows=350 width=28)
6	Workers Planned: 2
7	-> Partial HashAggregate (cost=1857935.50..1857937.25 rows=175 width=28)
8	Group Key: date.d_year, customer.c_nation
9	-> Hash Left Join (cost=43704.05..1852385.14 rows=555036 width=20)
10	Hash Cond: (lineorder.lo_orderdate = date.d_datekey)
11	-> Hash Join (cost=43607.51..1850829.23 rows=555036 width=20)
12	Hash Cond: (lineorder.lo_partkey = part.p_partkey)
13	-> Hash Join (cost=15119.92..1799026.48 rows=1547100 width=24)
14	Hash Cond: (lineorder.lo_custkey = customer.c_custkey)
15	-> Hash Join (cost=881.74..1675716.79 rows=7572690 width=20)
16	Hash Cond: (lineorder.lo_suppkey = supplier.s_suppkey)
17	-> Parallel Seq Scan on lineorder (cost=0.00..1576397.53 rows=37494753 width=24)
18	-> Hash (cost=806.00..806.00 rows=6059 width=4)
19	-> Seq Scan on supplier (cost=0.00..806.00 rows=6059 width=4)
20	Filter: ((s_region)::text = 'AMERICA'::text)
21	-> Hash (cost=12640.00..12640.00 rows=91935 width=12)
22	-> Seq Scan on customer (cost=0.00..12640.00 rows=91935 width=12)
23	Filter: ((c_region)::text = 'AMERICA'::text)
24	-> Hash (cost=23778.00..23778.00 rows=287007 width=4)
25	-> Seq Scan on part (cost=0.00..23778.00 rows=287007 width=4)
26	Filter: (((p_mfgr)::text = 'MFGR#1'::text) OR ((p_mfgr)::text = 'MFGR#2'::text))
27	-> Hash (cost=64.57..64.57 rows=2557 width=8)
28	-> Seq Scan on date (cost=0.00..64.57 rows=2557 width=8)

Figura 12 - Plano de execução da query Q4.1

Conclusão

Com este trabalho podemos concluir que as especificações das máquinas em que as queries correm têm um grande impacto no tempo que leva a estas serem processadas, podendo levar a diferenças de grandes magnitudes.

Este trabalho não nos permitiu ver qual o efeito que o uso de keys pode ter ao correr queries, uma vez que os tempos de processamento destas foram bastante semelhantes com e sem keys. Além disso, após análise dos planos de execução, podemos observar que as keys não foram propriamente utilizadas, o que reforça a falta de efeito que as keys tiveram ao correr as queries.

Dificuldades Encontradas

Devido a alterações relacionadas com o funcionamento do comando Copy e a sua compatibilidade entre o sistema operativo Windows10 e as versões mais recentes do postgreSQL (11+) não nos foi possível colocar os dados da tabela "lineorder" na base de dados. Isto acontece devido ao comando Copy chamar uma função fstat() que, sendo uma versão 32 bits, não suporta ficheiros superiores a 2GB. Para ultrapassar esta dificuldade optámos por utilizar a versão de postgreSQL 10.

Anexo

Lista de Queries:

Q1.1:

```
select sum(lo_revenue) as revenue
from lineorder
left join date on lo_orderdate = d_datekey
where d_year = 1993
and lo_discount between 1 and 3
and lo_quantity < 25;
```

Q1.2:

```
select sum(lo_revenue) as revenue
from lineorder left join date on lo_orderdate = d_datekey
where d_yearmonthnum = 199401
and lo_discount between 4 and 6
and lo_quantity between 26 and 35;
```

Q1.3:

```
select sum(lo_revenue) as revenue
from lineorder left join date on lo_orderdate = d_datekey
where d_weeknuminyear = 6 and d_year = 1994 and
lo_discount between 5 and 7 and lo_quantity between 26 and
35;
```

Q2.1

select sum(lo_revenue) as lo_revenue, d_year, p_brand1 from lineorder left join date on lo_orderdate = d_datekey left join part on lo_partkey = p_partkey left join supplier on lo_suppkey = s_suppkey where p_category = 'MFGR#12' and s_region = 'AMERICA' group by d_year, p_brand1 order by d_year, p_brand1;

Q2.2

select sum(lo_revenue) as lo_revenue, d_year, p_brand1 from lineorder left join date on lo_orderdate = d_datekey left join part on lo_partkey = p_partkey left join supplier on lo_suppkey = s_suppkey where p_brand1 between 'MFGR#2221' and 'MFGR#2228' and s_region = 'ASIA' group by d_year, p_brand1 order by d_year, p_brand1;

Q2.3

select sum(lo_revenue) as lo_revenue, d_year, p_brand1 from lineorder left join date on lo_orderdate = d_datekey left join part on lo_partkey = p_partkey left join supplier on lo_suppkey = s_suppkey where p_brand1 = 'MFGR#2239' and s_region = 'EUROPE' group by d_year, p_brand1 order by d_year, p_brand1;

Q3.1

select c_nation, s_nation, d_year, sum(lo_revenue) as lo_revenue from lineorder left join date on lo_orderdate = d_datekey left join customer on lo_custkey = c_custkey left join supplier on lo_suppkey = s_suppkey where c_region = 'ASIA' and s_region = 'ASIA'and d_year >= 1992 and d_year <= 1997 group by c_nation, s_nation, d_year order by d_year asc, lo_revenue desc;

Q3.2

select c_city, s_city, d_year, sum(lo_revenue) as lo_revenue from lineorder left join date on lo_orderdate = d_datekey left join customer on lo_custkey = c_custkey left join supplier on lo_suppkey = s_suppkey where c_nation = 'UNITED STATES' and s_nation = 'UNITED STATES' and d_year >= 1992 and d_year <= 1997 group by c_city, s_city, d_year order by d_year asc, lo_revenue desc;

Q3.3

select c_city, s_city, d_year, sum(lo_revenue) as lo_revenue from lineorder left join date on lo_orderdate = d_datekey left join customer on lo_custkey = c_custkey left join supplier on lo_suppkey = s_suppkey where (c_city='UNITED KI1' or c_city='UNITED KI5') and (s_city='UNITED KI1' or s_city='UNITED KI5') and d_year >= 1992 and d_year <= 1997 group by c_city, s_city, d_year order by d_year asc, lo_revenue desc;

Q3.4

select c_city, s_city, d_year, sum(lo_revenue) as lo_revenue from lineorder left join date on lo_orderdate = d_datekey left join customer on lo_custkey = c_custkey left join supplier on lo_suppkey = s_suppkey where (c_city='UNITED KI1' or c_city='UNITED KI5') and (s_city='UNITED KI1' or s_city='UNITED KI5') and d_yearmonth = 'Dec1997' group by c_city, s_city, d_year order by d_year asc, lo_revenue desc;

Q4.1

select d_year, c_nation, sum(lo_revenue) - sum(lo_supplycost) as profit from lineorder left join date on lo_orderdate = d_datekey left join customer on lo_custkey = c_custkey left join supplier on lo_suppkey = s_suppkey left join part on lo_partkey = p_partkey where c_region = 'AMERICA' and s_region = 'AMERICA' and (p_mfgr = 'MFGR#1' or p_mfgr = 'MFGR#2') group by d_year, c_nation order by d_year, c_nation;

Q4.2

select d_year, s_nation, p_category, sum(lo_revenue) - sum(lo_supplycost) as profit from lineorder left join date on lo_orderdate = d_datekey left join customer on lo_custkey = c_custkey left join supplier on lo_suppkey = s_suppkey left join part on lo_partkey = p_partkey where c_region = 'AMERICA'and s_region = 'AMERICA' and (d_year = 1997 or d_year = 1998) and (p_mfgr = 'MFGR#1' or p_mfgr = 'MFGR#2') group by d_year, s_nation, p_category order by d_year, s_nation, p_category;

Q4.3

select d_year, s_city, p_brand1, sum(lo_revenue) - sum(lo_supplycost) as profit from lineorder left join date on lo_orderdate = d_datekey left join customer on lo_custkey = c_custkey left join supplier on lo_suppkey = s_suppkey left join part on lo_partkey = p_partkey where c_region = 'AMERICA'and s_nation = 'UNITED STATES' and (d_year = 1997 or d_year = 1998) and p_category = 'MFGR#14' group by d_year, s_city, p_brand1 order by d_year, s_city, p_brand1;

Lista de Keys:

1:

```
ALTER TABLE date
ADD PRIMARY KEY(d_datekey);
```

2:

```
ALTER TABLE supplier
ADD PRIMARY KEY(s_suppkey);
```

3:

```
ALTER TABLE customer
ADD PRIMARY KEY (c_custkey);
```

4:

```
ALTER TABLE part
ADD PRIMARY KEY (p_partkey);
```

5:

```
ALTER TABLE lineorder
ADD PRIMARY KEY (lo_orderkey, lo_linenumber);
```

6:

```
ALTER TABLE lineorder
ADD FOREIGN KEY (lo_orderdate) REFERENCES date (d_datekey);
```

7:

```
ALTER TABLE lineorder
ADD FOREIGN KEY (lo_commitdate) REFERENCES date (d_datekey);
```

8:

```
ALTER TABLE lineorder
ADD FOREIGN KEY (lo_suppkey) REFERENCES supplier (s_suppkey);
```

9:

```
ALTER TABLE lineorder
ADD FOREIGN KEY (lo_custkey) REFERENCES customer (c_custkey);
```

10:

```
ALTER TABLE lineorder
ADD FOREIGN KEY (lo_partkey) REFERENCES part (p_partkey);
```